

Diskarte

The Organizational Runtime

What it is, why it exists, how it works, what has been built, and what comes next — the canonical technical overview, written for engineers.

How to read this document

Diskarte’s major architectural boundaries are either **enforced in implementation**, **established in approved doctrine**, or **explicitly tracked as implementation awaiting ratification**. This deck never blurs those states. Every capability claim carries one of five status chips:

○ VISION

○ ARCHITECTURALLY COMPLETE

● IMPLEMENTED

● DEPLOYED · INTERNAL DEV

○ PLANNED

Vision	A named direction. Neither designed in detail nor ruled.
Architecturally Complete	Ruled in approved architecture. The design is closed; the code does not exist yet.
Implemented	Built, committed, and proven through available verification. Does not imply deployment.
Deployed · Internal Dev	Serving today — in Diskarte’s internal development environment only. Nothing customer-facing is production-deployed, by intent.
Planned	On the roadmap behind a recorded gate — never a date.

An organizational runtime, not another agent

Diskarte is a constitutional operating environment in which an organization's commitments, work, and knowledge are governed — and through which it can safely delegate increasing autonomy to AI.

Agents may operate inside it. The product is the runtime around them: context, authority, governance, memory, judgment, and execution.

Evidence ≠ Truth ≠ Canon. Autonomy is earned through governance.

This exists because AI made execution cheap and left organizational judgment, context, and memory exactly where they were.

DISKARTE IS DELIBERATELY NOT

An AI assistant — it governs organizations; it does not chat for their members.

An agent framework — agents are performers inside seats, gated like every other occupant.

A workflow engine — work flows from authority, not from diagrams of tasks.

A knowledge base or enterprise search — memory is an epistemic state with named blind spots, not a filing cabinet.

A model wrapper — the constitution can never contain a model or vendor name. The schema refuses it.

AI made execution fast. It did not make organizations coherent.

Why current AI systems fail inside organizations, and what layer is missing.

Where AI fails inside organizations

01 · CONTEXT

Tools know tasks. They do not know the organization.

AI drafts, summarizes, and executes — without knowing why the work matters, who holds authority, or which constraints cannot be violated.

02 · JUDGMENT

More output creates more undelegated judgment.

Without governed escalation, acceleration is noise — pushing more cognitive load onto exactly the person it was meant to relieve.

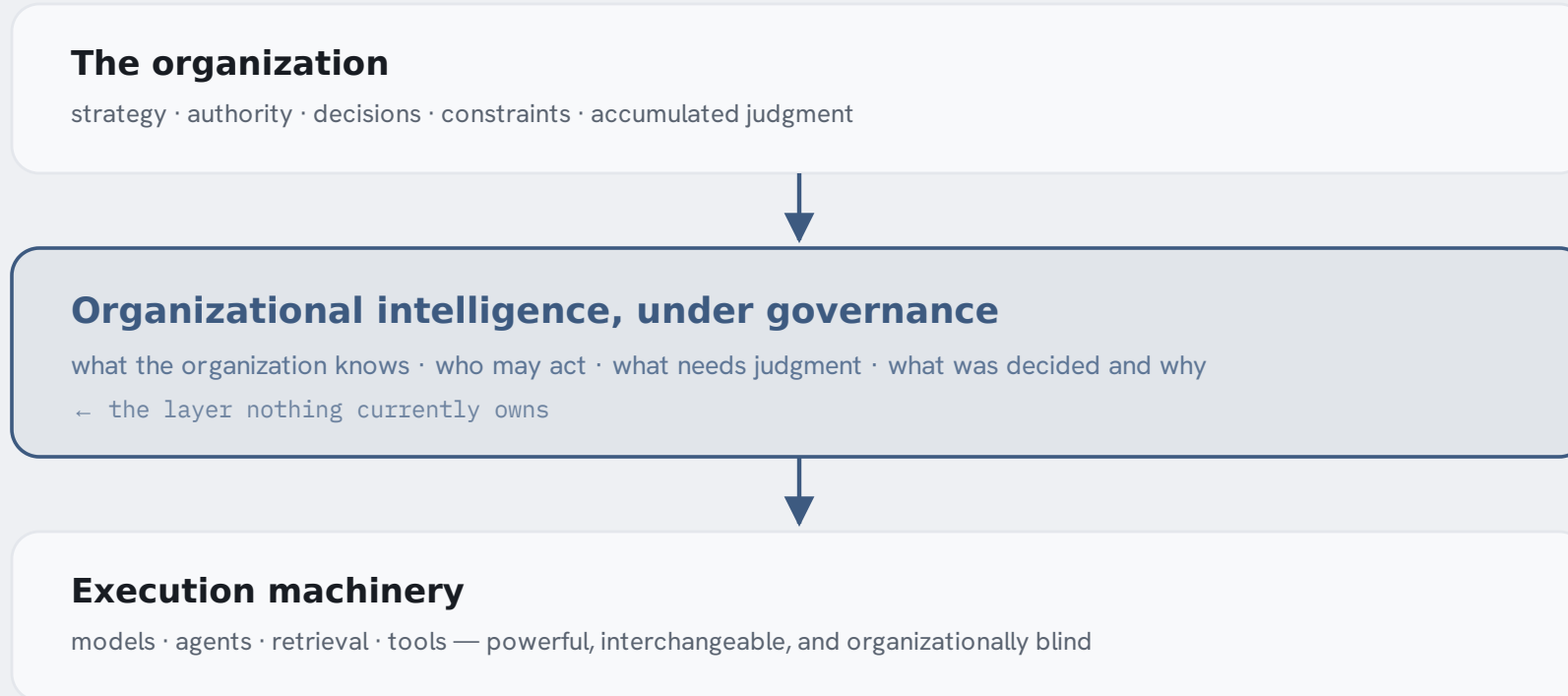
03 · MEMORY

Organizations keep relearning what they already knew.

Decisions lose their rationale; projects lose context; understanding is rebuilt every time people, tools, or priorities change.

Assistants, agent frameworks, and retrieval systems each attack one symptom. None of them owns the organization's **authority, judgment, or memory** — so none can fix the underlying failure.

The missing layer



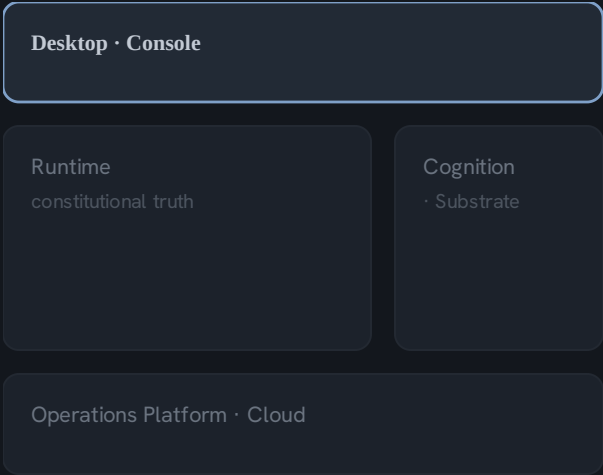
Diskarte is this layer, built as a runtime: organizational knowledge and authority above, replaceable execution below, and governance at every crossing between them.

Autonomy is not a model property. It is a governance property.

An organization can only delegate work it can authorize, verify, and remember. Diskarte exists to make those three verbs machine-legible — so delegation to AI can grow with evidence instead of faith. Intelligence proposes; governance disposes; humans remain the only source of canonical authority.

What a founder actually touches.

The Desktop, the Morning Brief, memory, judgment, and practices — every claim in this part carries its implementation status.



The system at a glance



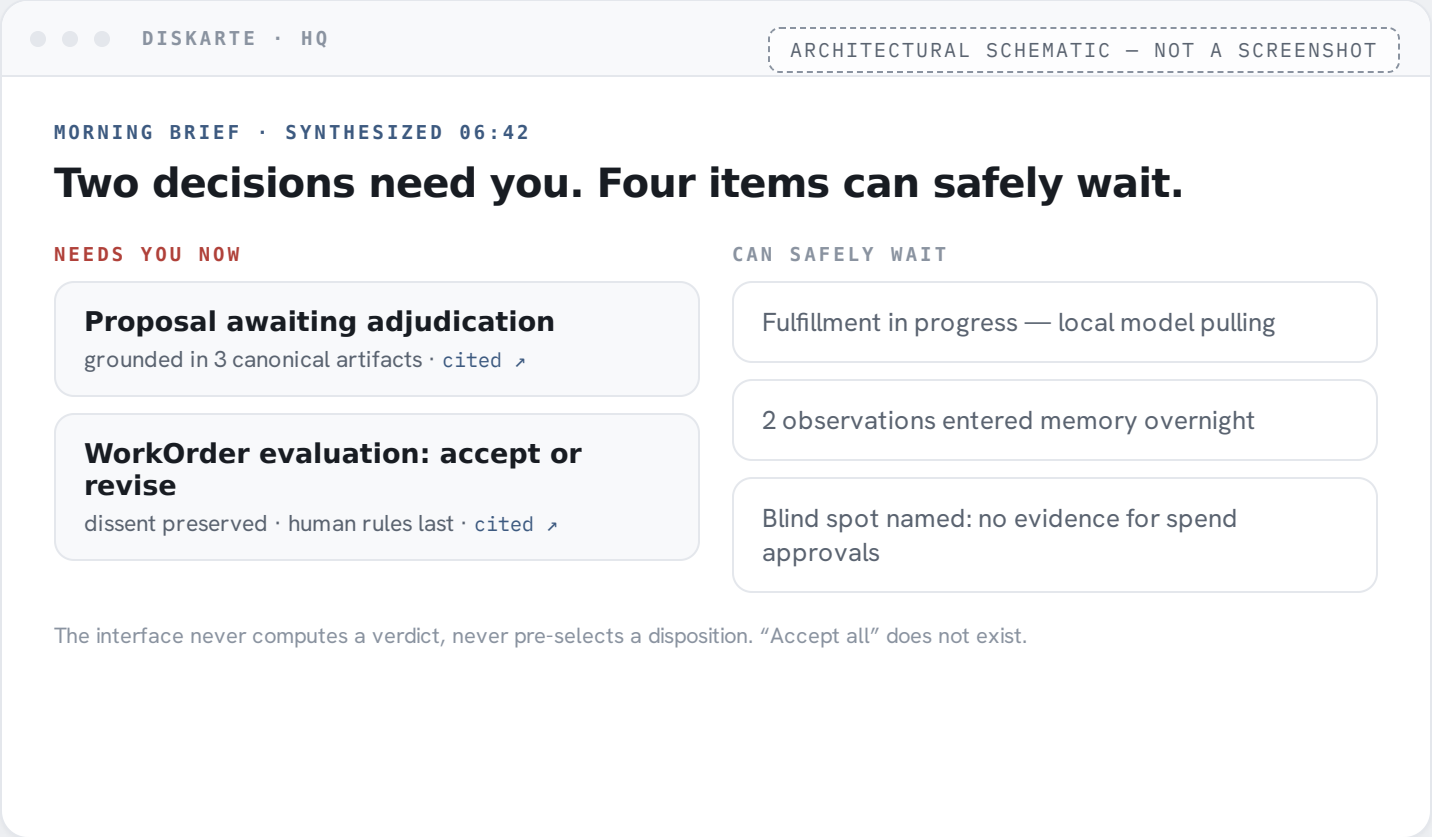
One seam in: every surface speaks to the Runtime through a single Platform API. Nothing reaches the kernel, storage, or cognition directly.

The Morning Brief

The founder’s day starts from a synthesized brief: what needs judgment now, and what can safely wait.

Every line is grounded — the composer refuses items without citations. Attention is ranked by organizational consequence, not recency.

● IMPLEMENTED



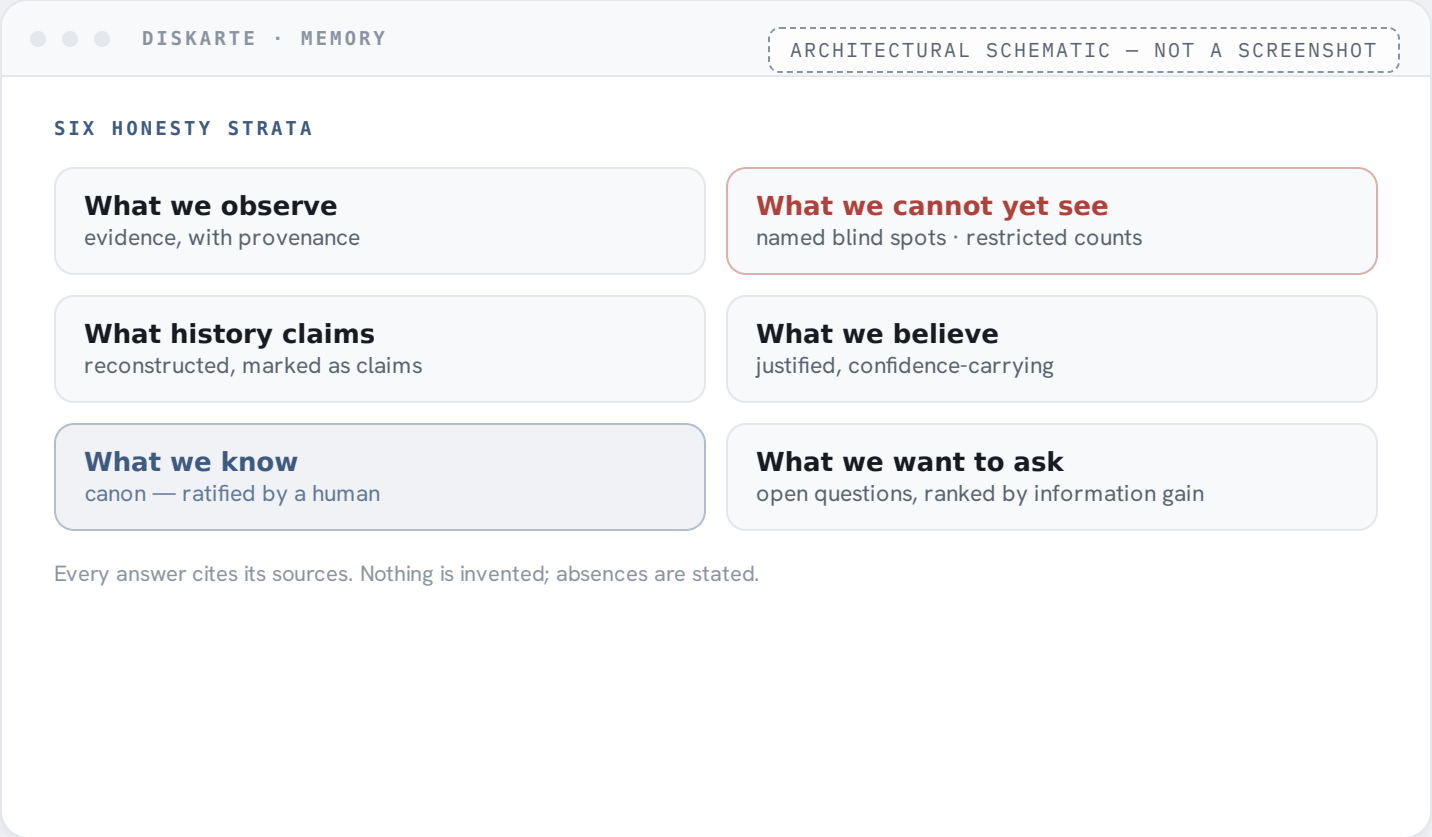
This exists because the scarce resource in an AI-fast organization is founder attention — so the product spends machine cycles to conserve it.

Organizational Memory

Memory is an epistemic state, not a filing cabinet. The organization can always answer: what do we observe, believe, know — and what can we not yet see?

Blind spots are first-class output, shown above the fold. What cannot be observed is named, never inferred.

● IMPLEMENTED



This exists because a memory that cannot distinguish belief from knowledge — or name its own gaps — will eventually govern from fiction.

Organizational Judgment

Judgment is distinct from memory: two governed loops, both terminating in a human ruling.

Work judgment — accept, revise with direction, or escalate every produced deliverable. Dissent is preserved, never flattened.

Belief judgment — candidate understanding earns standing through rulings before it may inform anything.

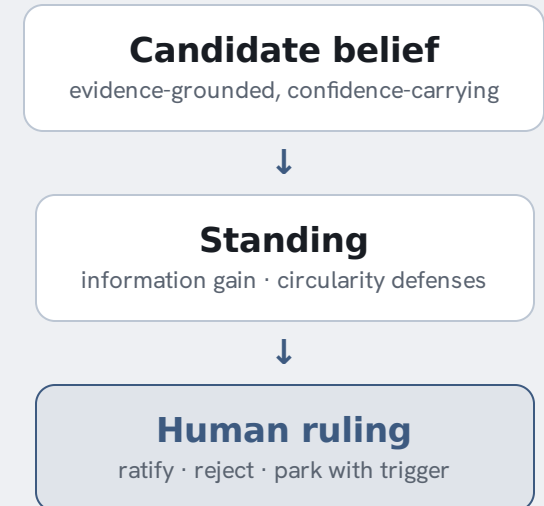
● IMPLEMENTED

DETERMINISTIC LAYER

WORK LOOP



BELIEF LOOP



The UX law: the interface never computes a verdict, never pre-selects a disposition — and batch-accept does not exist anywhere in the product.

Capabilities and Practices

Adopting a capability is hiring a profession, not installing a plugin.

A **Capability** is portable professional knowledge — it arrives with standards and defers to your law. A **Practice** is your organization's sovereign derivation of it: installed through a ruling, tuned by your dispositions, earning standing through judged work.

● IMPLEMENTED

AUTHORING KIT + PRACTICE SURFACES

This exists because professional knowledge should be distributable — while how **your** organization works must remain yours alone. Distribution never substitutes for the sovereign act.

Capability package

portable knowledge · declares abstract requirements · publisher-provenance

↓ **install is a ruling ceremony — values may stay deliberately unset**

Practice

the organization's governed instance — diverges freely, cites its law

↓ **work executes through seats, verified the same for every occupant**

Precedent

judged work accrues; corrections decay repeated mistakes

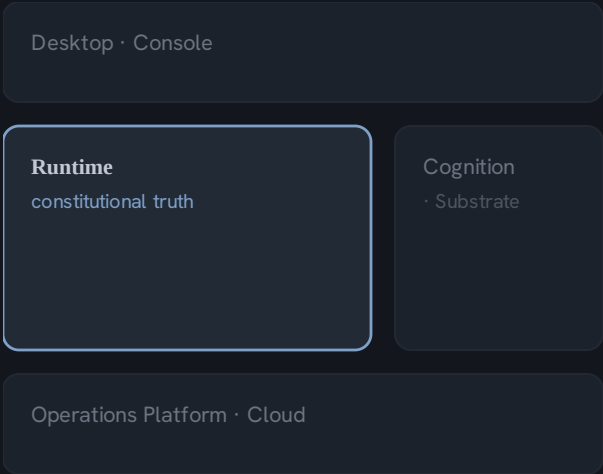
Product surfaces, honestly labeled

SURFACE	WHAT IT DOES	STATUS
Claim & onboarding ceremony	Setup code → identity → preview → human confirm → custody → sign-in	● IMPLEMENTED DEPLOYMENT PENDING
Morning Brief + attention queue	Grounded daily synthesis; consequence-ranked judgment queue	● IMPLEMENTED
Organizational Memory	Six honesty strata; blind spots first; cited answers	● IMPLEMENTED
Judgment loops (work + belief)	Human rulings over executed work and candidate understanding	● IMPLEMENTED
Practice library + install sitting	Capability adoption as a governed ruling	● IMPLEMENTED
Brownfield import	Folder → bundle → dry-run → import → healing → review	● IMPLEMENTED OFFLINE HALF
Substrate profile selection screen	Question form → profile cards → explicit adoption	○ PLANNED DATA PATH LIVE; SCREEN IS PRESENTATION
Production / operations domains	Building, shipping, and observing software through the runtime	○ VISION

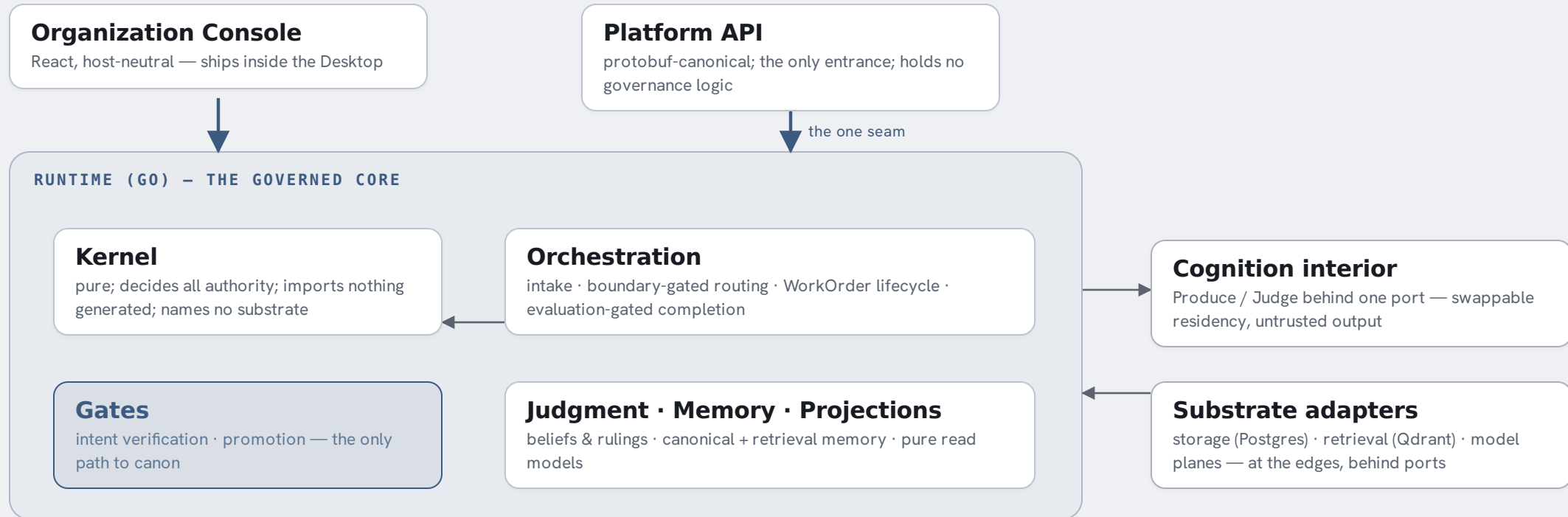
Diskarte today can host an organization’s **decisions, memory, and governed work**. The deep execution domains — production, deployment, operations of the org’s own products — are the roadmap, and are labeled as such.

Truth lives where it can be enforced.

The anatomy of the Runtime — what each layer owns, and what it must never absorb.



Runtime anatomy



Deployment simplifies; architecture does not. All of this can run as one local binary — fewer processes, same durable boundaries.

The kernel: seven primitives, nothing else

Actor

an addressable participant with stable identity — the founder's Boundary is the root of all authority

Role

a durable set of responsibilities; data, not code

Boundary

what an actor may do without escalation — one evaluation concept, three verdicts: permitted · requires-approval · forbidden

Decision

a choice that passed governance; rationale and dissent preserved as first-class content

WorkOrder

the governed unit of work; cannot complete except through Evaluation

Artifact

a durable record with type, authority, provenance — canon is the set of canonical Artifacts

Evaluation

typed assessment against a rubric: accept · revise · escalate — the only way work is done

— and no eighth

five admission criteria; no second instance of any primitive's concern, ever

WHAT THE KERNEL NEVER DOES

It never reasons, never names a substrate, never stores authority as a fact, never imports generated code, never flattens dissent, and never declares completion without Evaluation. **It governs; it does not think.**

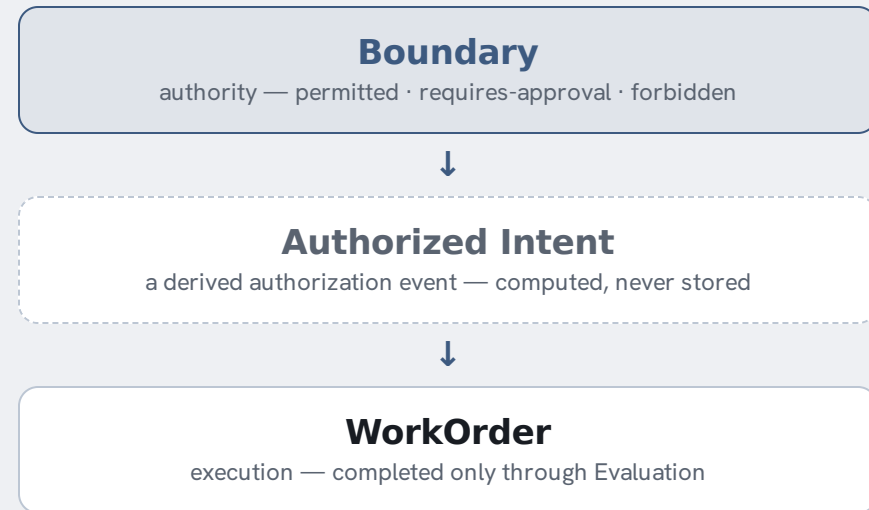
Authority is computed, never stored

```
Authority(actor, context)
  = Role(actor) n Boundary(actor)
```

...evaluated at the moment of authorization. Permissions are assigned; authority is **derived** from responsibility and limit.

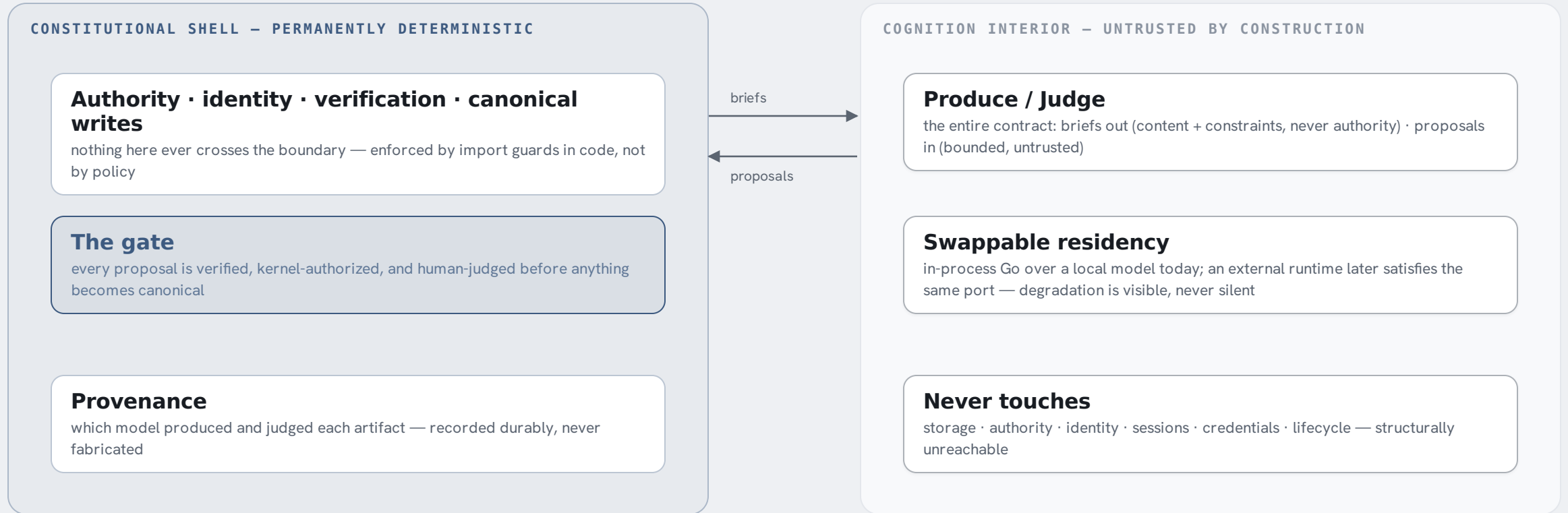
Three sources of authorization — a founder command, delegated authority within Boundary, or a formal Decision — all pass through the **same** Boundary evaluation. In-Boundary work needs no committee: Diskarte is governed by boundaries, not mandatory deliberation.

THE SPINE — EVERY GOVERNED SIDE EFFECT CROSSES IT



Routing is dispatch, never authority. Anything that “routes” decides where already-authorized work goes — a second who-may-do-what concept is the failure mode this kernel exists to refuse.

Cognition is a domain boundary



It reasons; it never governs. The kernel decides all authority and never reasons; cognition reasons and never decides. The seam between them is the product.

Two memories, never conflated

CANONICAL MEMORY

The system of record

The set of canonical Artifacts — the organization reconstructs from it

Durable, governed, evaluation-gated

Written only through verification + kernel authorization

If lost: catastrophic

RETRIEVAL MEMORY

The acceleration layer

Embeddings and indexes that ground cognition

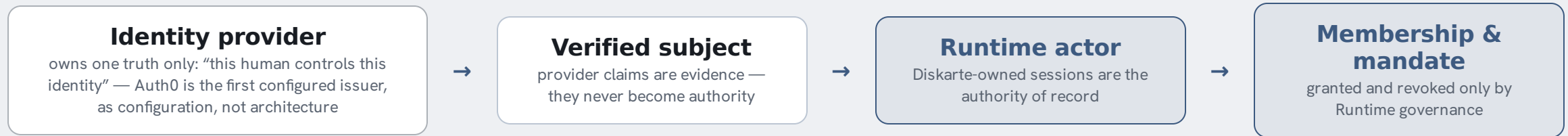
Derived, advisory, lossy

Reconstructible from canonical memory at any time

If lost: re-index

Collapsing them is prohibited. A vector index that becomes a source of truth is a second system of record — the exact failure this split exists to prevent.

Identity is infrastructure; authority is governance



FOUR PROOFS, FOUR GRANTERS

Possession is not identity. Identity is not intent. Intent is not installation legitimacy. Legitimacy is not authority. **Any shortcut that lets one proof imply the next is the threat model.**

Every truth has exactly one owner

LAYER	OWNS	MUST NEVER ABSORB
Kernel	authority decisions, invariants	reasoning, substrate names, generated code
Runtime governance	canon, membership, policies, bindings	anything a model asserted but no human ratified
Cognition	proposals, evaluations, learning	authority, canon writes, credentials, identity
Desktop	custody — what is on this disk; the machine’s evidence	server truths; it reflects them, visibly labeled
Operations Platform	consumption of setup codes, installations, releases, appliances	constitutional authority in any organization
Identity provider	the external identity itself	any decision about what that identity may do
Substrate layer	providers, resources, health	knowledge — replacing a provider never mutates a Practice
Nobody	composite states — “ceremony complete”, “ready to execute” — are derived by asking each plane, never stored. There is no orchestrator of record, and there never will be.	

What you cannot enforce, you may only reflect — and reflections are visibly labeled as reflections.

The boundaries, made physical

Diskarte is a federation of repositories. Every cross-repo dependency is a versioned, published artifact — never a source import — so an architectural violation becomes a build failure, not a review comment.

runtime

deterministic truth: kernel, contracts, Go runtime, the constitutional corpus

ui

the interaction model: console, design system, platform client

desktop

native host and the composition point — pins runtime + UI by digest, fails closed on mismatch

cognition

advisory intelligence — chartered before its first model line; structurally barred from canon

dev

the Operations Platform — how the company operates itself

cloud

infrastructure only: deploys immutable digests, never compiles product source

canon

private authority: ruled architecture and doctrine — when code and canon disagree, code is the drift

site

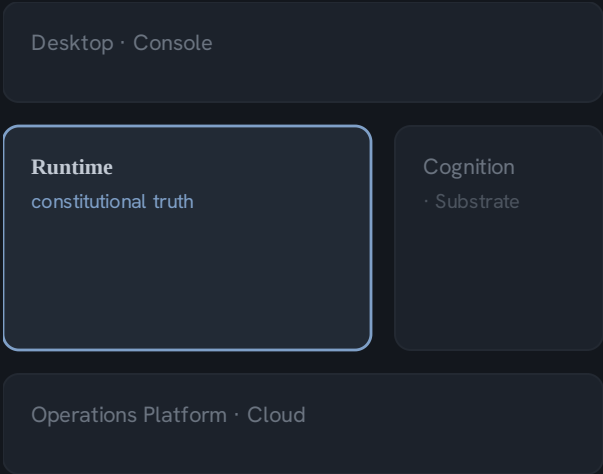
the public surface, diskarte.jp

The Desktop is the composition point: it owns the manifest of what runs together. Freshness arrives only through an explicit, founder-authorized re-pin.

Evidence is not truth, and evidence is not canon.

The runtime builds the strongest justified model from evidence —
and nothing governs until a human ratifies it into canon.

YOU ARE HERE



Three layers, three different things

Evidence

what the organization can observe — always partial, often stale, sometimes performative



Truth a justified model

the strongest model the evidence supports — confidence-carrying, contestable, never certainty

↓ **only through human ratification**

Canon

what the organization has committed to — the only layer governance may act on

Evidence is not truth. Exhaust is lossy and biased; observation has boundaries; what cannot be seen is named, never inferred.

Truth is not canon. A justified model is a set of claims with confidence and provenance — a question well asked, not a commitment.

Canon is not reality. It is what the organization decided. Reality never appears as a runtime object — only as the hidden grader in benchmarks.

FULL PIPELINE (IMPLEMENTATION): EVIDENCE → INFERENCE → PROPOSAL → RATIFICATION → GOVERNED CANON

The generating rule: the runtime operates on exactly two kinds of thing — observable evidence and governed commitment. It never operates on reality.

Why the separation is load-bearing

THE GOODHART ARGUMENT

The moment evidence drives governance, evidence becomes a target — and targets get gamed. So evidence can never be self-certifying, and the promotion from model to canon must be a **governed act**, not a computation.

INFLUENCE VS AUTHORITY

The runtime may **observe** influence — who actually shapes outcomes. It may **act** only from authority that exists in canon. An observed authority pattern is a ratification question, never a warrant to govern.



Three edges most systems get wrong, all refused here: evidence $\nRightarrow$ canon · influence $\nRightarrow$ authority · drift $\nRightarrow$ silent canon mutation.

Two constitutions, two altitudes

KERNEL CONSTITUTION · 7 PRIMITIVES

How any single act is authorized

Actor, Role, Boundary, Decision, WorkOrder, Artifact, Evaluation. Binding for the kernel layer; names no substrate; unchanged since its ratification.

● IMPLEMENTED

ORGANIZATIONAL MODEL · 13 PRIMITIVES

How an organization knows and commits

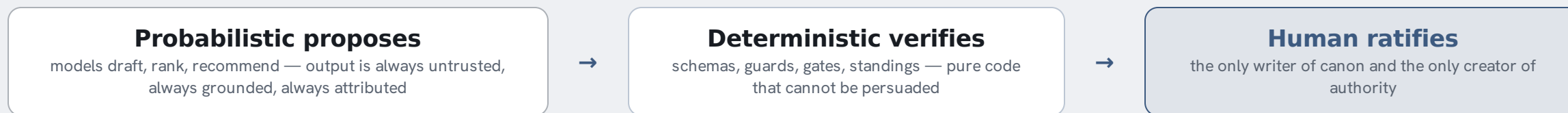
Evidence, Proposal, Genesis, Canon, Policy, Readiness, Consensus, Drift and its classification, Influence-vs-Authority, Epoch, Integrity, Crisis. A nine-stage state model connects them.

○ ARCHITECTURALLY COMPLETE PARTIALLY REALIZED · FORMAL STATUS PENDING

They meet at exactly one seam: **Governance Readiness → the kernel’s authorization spine.** Consensus writes authority *into* canon; the kernel derives per-action authority *from* canon. Realization varies by primitive — several are fully live, some partial, and the deepest (Integrity, Constitutional Crisis) are deliberately future. The full realization matrix is in the appendix.

Do not read these as one list: **seven govern acts; thirteen govern knowledge.** Different layers, deliberately.

Deterministic before probabilistic



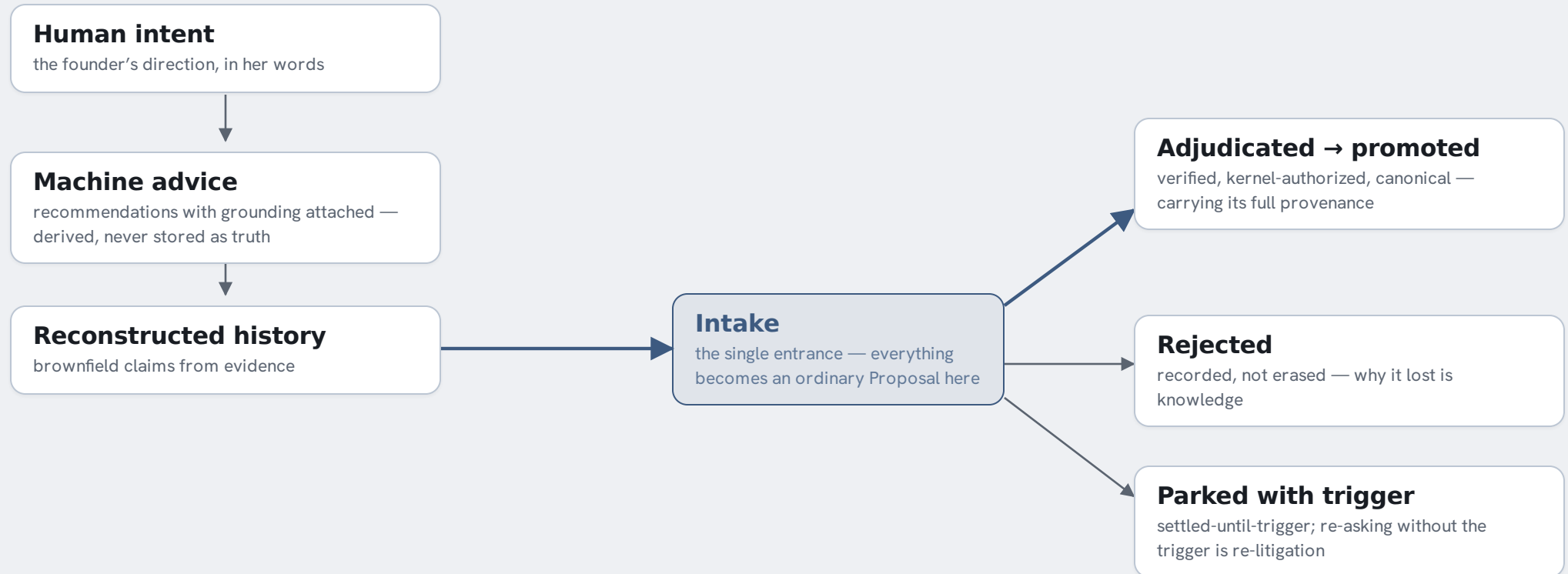
Learning starts immediately; trust is earned; authority is governed. Learners may observe and predict from day zero — privately. Promotion criteria are ratified *before* any candidate exists, so grading is never motivated.

A trust grant is a governed, revocable record with its own regression rule. None has been granted yet — the first will be evidence-backed, and that is the point.

Advisory quality can improve 100× with zero ontology change. Better models simply produce better-evidenced proposals through the same door.

ML never decides. It recommends into governed gates — at every stage, at every altitude, permanently.

One door into the constitution



No other write path exists. Machine advice enters through the same door as human intent, is adjudicated by the same governance, and executes through the same pipeline. No platform auto-apply, ever; batch-accept does not exist.

Authority strata and delegation

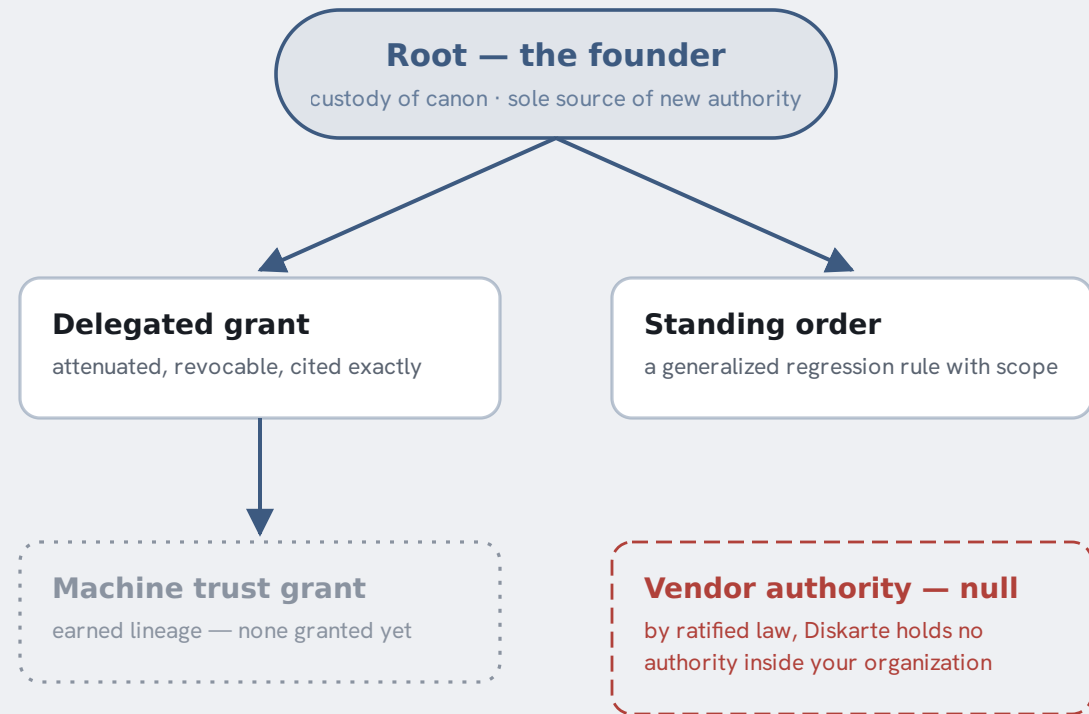
An organization has exactly one root-authority lineage. Everything else is delegation under it — never multiplication of roots.

Declared Reach: no ruling binds beyond its declared scope, force, mortality, and conveyability. Silence means instance-only, terminal, manual.

Ratification-Allocation: humans alone create, widen, or amend authority. A mechanism may only allocate within a pre-ratified extent — and must refuse anything requiring novel judgment back into a human proposal.

○ ARCHITECTURALLY COMPLETE

RULED CHARTER · IMPLEMENTATION EVIDENCE-GATED



Machine ratification never exists. Trust grants are the earned machine lineage of delegation — bounded, evidenced, revocable, and reversible by their own regression rules.

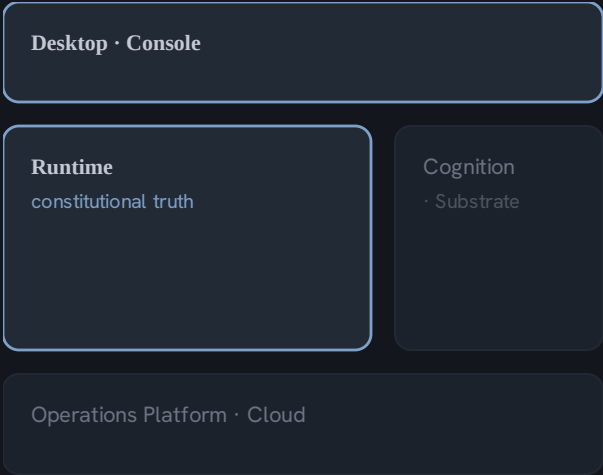
The Runtime owns constitutional truth because only it can revoke.

Operations can observe an organization; the Desktop can custody its files; a catalog can advise it; a model can draft for it. None of them can enforce a mandate, expire a session, or unwind an authority — so none of them may own one. The same pattern — a structural contract, a governed authorization, and evidence evaluated by nothing — repeats in every domain of the system: claims, substrate, and advice.

From a setup code to a governed organization.

The Greenfield Alpha — the gate called GF-0 — closed with the complete journey implemented and proven: install, claim, assess, recommend, adjudicate, activate, execute.

YOU ARE HERE



The bootstrap journey — one truth owner per edge

1 · DESKTOP + OPERATIONS

Install & claim

setup code → identity → confirm → custody. A legitimate installation.

2 · RUNTIME

Sign in

a Runtime actor exists; the sovereign container is founded — still empty.

3 · HUMAN

Answer questions

a few high-level answers — intake evidence, not a stored “intent.”

4 · DESKTOP

Environment Report

the machine, observed — evidence that authorizes nothing.

5 · ADVISORY LAYER

Recommendations

a deployment profile + starting points, grounded in the evidence — never applied.

6 · GOVERNANCE

Intake & adjudication

the founder rules on each recommendation through the one door.

7 · RUNTIME

Activation

the composition point — the constitution is populated as adjudicated acts.

8 · SUBSTRATE

Fulfillment → first session

async provisioning below the constitution; ordinary operation begins.

The machine is assessed by the **Desktop** · the profile is chosen by the **founder** · the organization is founded by the **Runtime** · every recommendation is **evidence** **the founder adjudicates**. No plane assesses, chooses, *and* founds — each does exactly one.

The claim ceremony: four independent authorities

IDENTITY PROVIDER

Proves the human

owns the one truth nobody else can mint:
this human controls this identity

OPERATIONS

Binds possession to identity

consumes the setup code exactly once,
inside one transaction, leaving evidence

DESKTOP

Custodies what was granted

the only plane that can own “what is on
this disk” — atomic, crash-safe commit

RUNTIME

Grants authority

sessions, membership, mandates —
because only it can revoke them

A chain of independent authorities, not a workflow. No plane vouches for another; each verifies for itself. “Ceremony complete” is derived by asking each plane its own truth — there is no ceremony object, no orchestrator database, and there never will be.

The ceremony ends where the product begins: a completed claim confers legitimacy — never capability, never authority, never resources.

The claim protocol

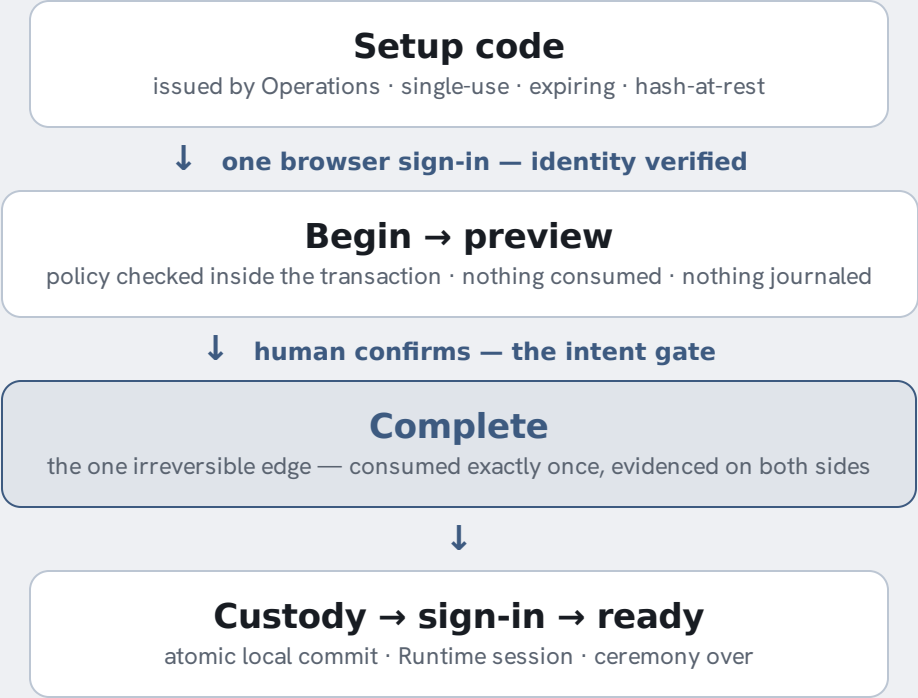
Looking is free; claiming is deliberate; the wire tells attackers nothing.

Begin consumes nothing. It verifies identity and policy, then returns a preview — so crashes and second thoughts cost nothing.

The intent gate. Complete is only ever crossed by a deliberate human confirmation on the rendered preview. No auto-complete path exists.

One dead answer. Every refusal is byte-identical outward; the full record is honest inward.

Three recovery actions, total: try again · sign in again · request a new code. When in doubt, reissue — recovery mints new legitimacy, never edits truth.



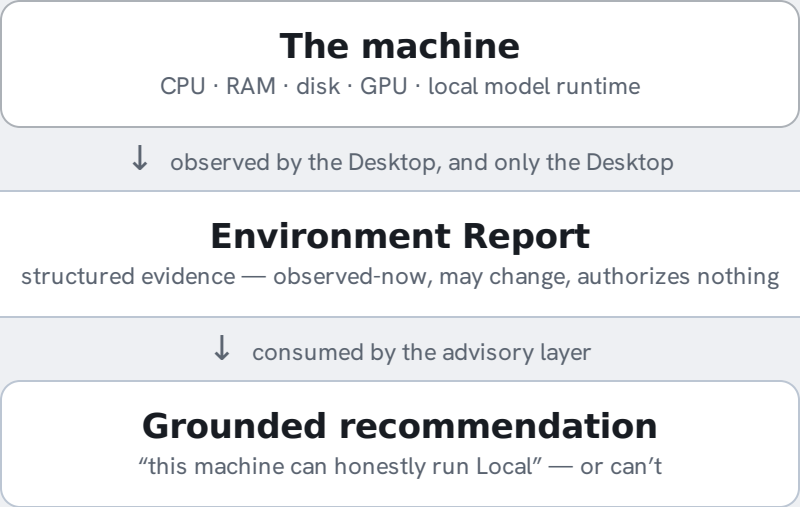
● IMPLEMENTED Proven end-to-end against the real stack, including a real-tenant identity proof. Public deployment is the next gate — FOUNDER-GATED

The Environment Report

Only the Desktop can see the hardware — so it observes the machine and produces evidence.

Coarse facts only: compute class, memory, free storage, whether a local model runtime is present. **Secret-free by construction** — never a serial number, hostname, username, or file listing. Anything unobservable degrades honestly to “unknown.”

● IMPLEMENTED



THE RUNTIME NEVER SEES THIS REPORT — IT IS NEVER ORGANIZATIONAL STATE

The machine’s analogue of brownfield exhaust: raw observation with epistemic status — the same evidence discipline, pointed at hardware.

Choosing a deployment shape, honestly

The advisory layer matches machine evidence and answers against the profile catalog — deterministically, entirely on the founder’s side.

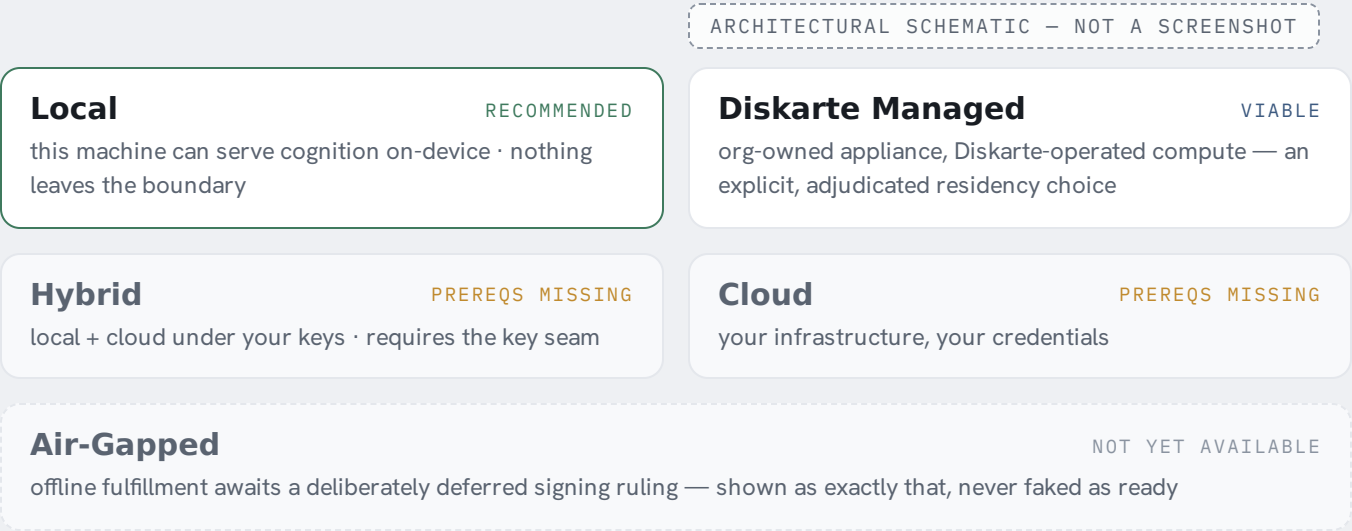
Each profile is ranked: recommended, viable, prerequisites missing, or not suitable — with the grounding shown. A browser with no machine probe honestly favors Managed, because Local cannot be assessed.

The founder **chooses**; nothing is ever auto-applied.

● IMPLEMENTED

SELECTION SCREEN: PRESENTATION PENDING

Suitability is computed, never stored — and ineligibility is stated with its reason. Honesty over conversion, structurally.



Activation is the composition point

Sign-in founds the sovereign **container**.

Activation populates its **constitution**.

Between the two, the organization exists but is nearly empty — by design. That is why a founder can answer questions “as” their organization before it has any practices.

Two provisionings, named apart: readying the machine (installation-scoped, may precede activation) and fulfilling the organization’s requirements (org-scoped, always after — before activation there is no organization to bind for).

Adopt the chosen profile

one governed act at activation

↓ decomposes — the profile object never enters the constitution

Org-owned substrate policy + bindings

residency decision · admitted provider kinds · one binding per abstract requirement

↓ async, restartable, below the constitution

Fulfillment

models pulled, resources provisioned — “ready” is derived, never stored

An organization can be **constitutionally complete with unsatisfied requirements** — visible as honest state, never as fake capability.

The launch gate checks legitimacy, identity, activation — never substrate. The desktop opens before any model finishes downloading.

First execution, fully accounted



THE TWO LAWS THIS CLOSES

Requests never reference models — a brief carries an abstract profile in; model identity travels as provenance out, enforced by guard tests.

“Which model produced this artifact” is durably answerable — producer and judge, recorded best-effort and honestly degraded, never fabricated.

● **IMPLEMENTED** Including a first-tenant proof: a managed appliance serving one organization's execution end-to-end, locally.

What the Greenfield gate proved — and what it deliberately did not do

SHIPPED & PROVEN, END TO END

The substrate seam — policy, bindings, abstract vocabulary

Durable execution provenance

The managed appliance control plane and its scheduler

The managed worker — deploy-ready behind a stable serving seam

Environment Report → recommendation → adoption, live in the desktop

Air-Gapped honesty — unavailability stated, never faked

● IMPLEMENTED

FULL CROSS-REPO VERIFICATION GREEN

DELIBERATELY NOT DONE — FOUNDER-GATED

First cloud deployment of the worker and claim gateway — infrastructure is written and validated; applying it is an explicit human act

Production model backend — serving runs on a deterministic backend today; a model backend is a configuration flip plus a licensing decision

The visual onboarding screen — the data path is live; the screen is presentation

No production license accepted, no billed infrastructure created, no live founder data touched

○ PLANNED

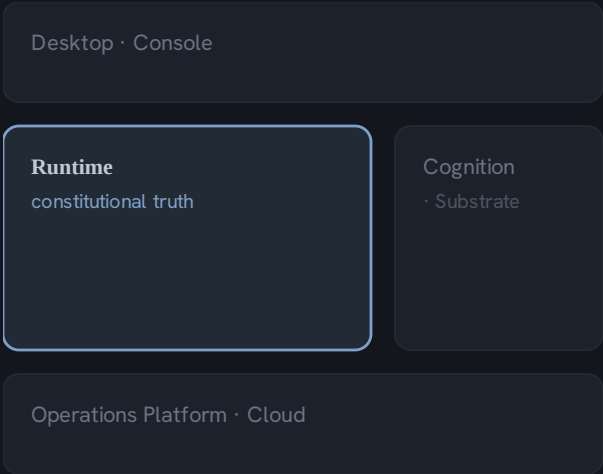
FOUNDER-GATED

This is the deliberate posture: engineering runs ahead and proves; deployment, spend, and licensing remain explicit human acts that CI cannot perform.

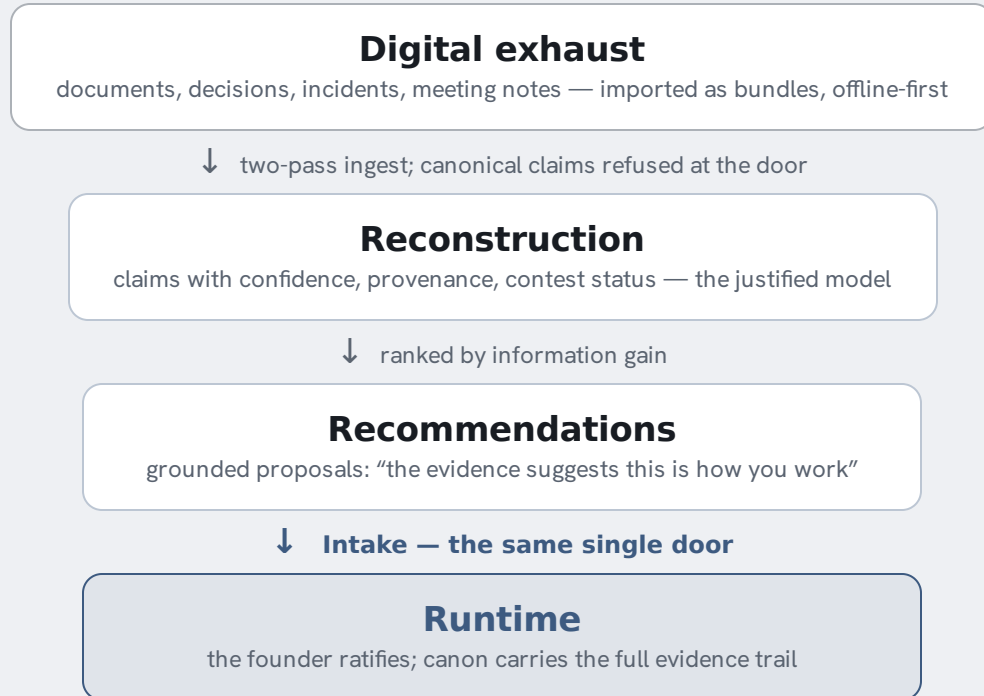
Organizations already exist. Their evidence does too.

Reconstructing an organization from its digital exhaust — honestly,
with named blind spots.

YOU ARE HERE



From exhaust to canon



Evidence is treated as what it is: lossy, biased, stale, sometimes performative. Confidence measures evidential support — never probability of truth.

Claims carry three orthogonal axes — what kind of signal (declared, observed, aspirational, historical), where in the pipeline, and what it claims about (structure, strategy, execution, authority, influence) — so the schema cannot even express a claim as truth.

Corrections outrank evidence. A human correction becomes a canonical claim; contradicting evidence later raises a reconciliation proposal — it never overwrites.

● **IMPLEMENTED** The offline half — bundles, ingest, claims, healing — is proven end-to-end. Live connectors to running systems are **GATED BY A RECORDED RULING**

Truthfulness is the feature

BLIND SPOTS, FIRST-CLASS

Verbal decisions, off-system approvals, mechanisms that leave no trace — the runtime says **“outside the observation boundary,”** with a reason, rather than fabricating a claim.

The review surface leads with what *cannot* be reconstructed — before showing what can.

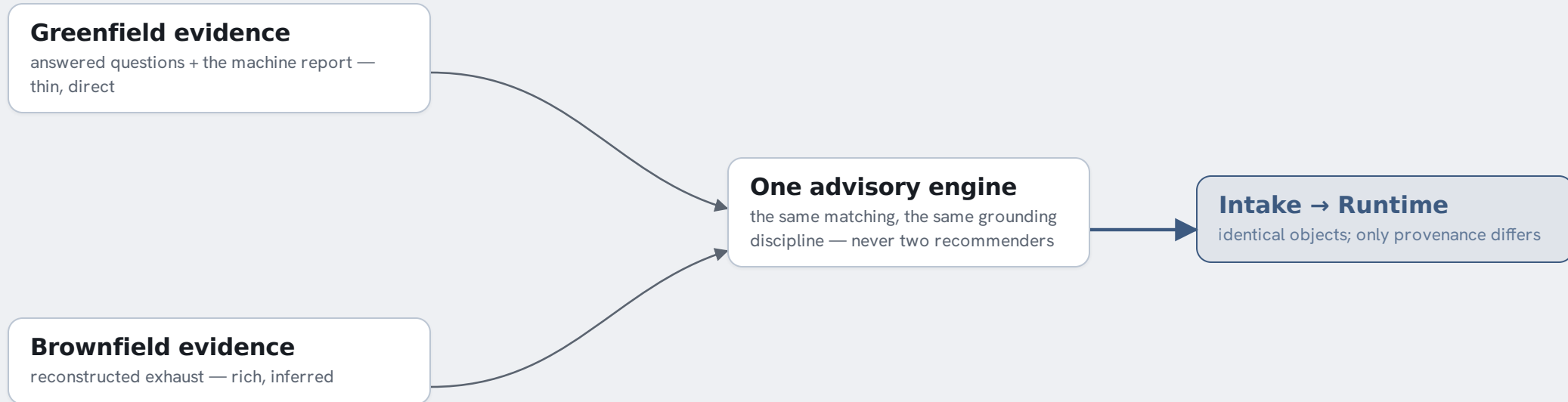
REPLAY HEALS

Import the same corpus twice: **zero new claims, N healed.** Ingest is idempotent by identity, so re-import repairs rather than duplicates.

This is the first “it actually understands” moment for a brownfield founder — produced by discipline, not by a model’s confidence.

No other product shows its blind spots. The magic is epistemic, never authoritative — and that is the trust story, not a limitation of it.

Greenfield and Brownfield converge

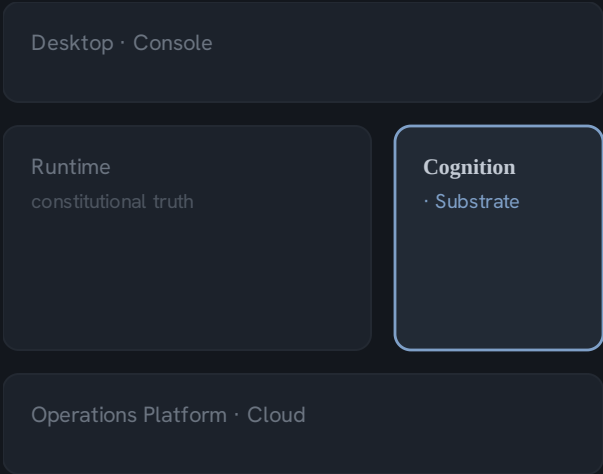


The convergence law: the Runtime never knows which path created an object. Evidence differs; the destination doesn't — and after bootstrap, the two paths are indistinguishable.

Foundations decompose, or they colonize.

How organizations start composed — without becoming instances of somebody else’s template.

YOU ARE HERE



A Foundation is a manifest, never an object

A Foundation — “the Sales Foundation,” “the Engineering Foundation” — is a versioned composition of capability packages and practice templates.

Adopting one **decomposes** it into ordinary constitutional acts through the normal governance pipeline. Afterward, the organization holds ordinary practices with adoption provenance — and could forget the Foundation ever existed, losing nothing.

“Which foundation are we on?” is a derived view over provenance — never a stored status.

○ ARCHITECTURALLY COMPLETE

Updates are recommendations. Full stop. A publisher’s v2 is a new catalog fact; the org’s advisory layer surfaces a grounded delta; divergence is sovereignty functioning.

Foundation manifest v2

capability packages + practice templates + suggested first proposals

↓ adoption — N ordinary adjudicated acts

The organization’s own practices

sovereign derivations — they diverge freely; the publisher loses all say



Adoption provenance

source ware · version · publisher — evidence, evaluated by nothing

“MANAGED FOUNDATIONS” — AUTO-TRACKING UPSTREAM — ARE BANNED BY NAME

Four verbs, four owners

Catalog recommends.

publishes wares and descriptors,
down to every organization

○ ARCH. COMPLETE

Runtime governs.

every adoption is an adjudicated
constitutional act

● IMPLEMENTED

Humans ratify.

recommendations render their
grounding; judgment is never
ceremonial

● IMPLEMENTED

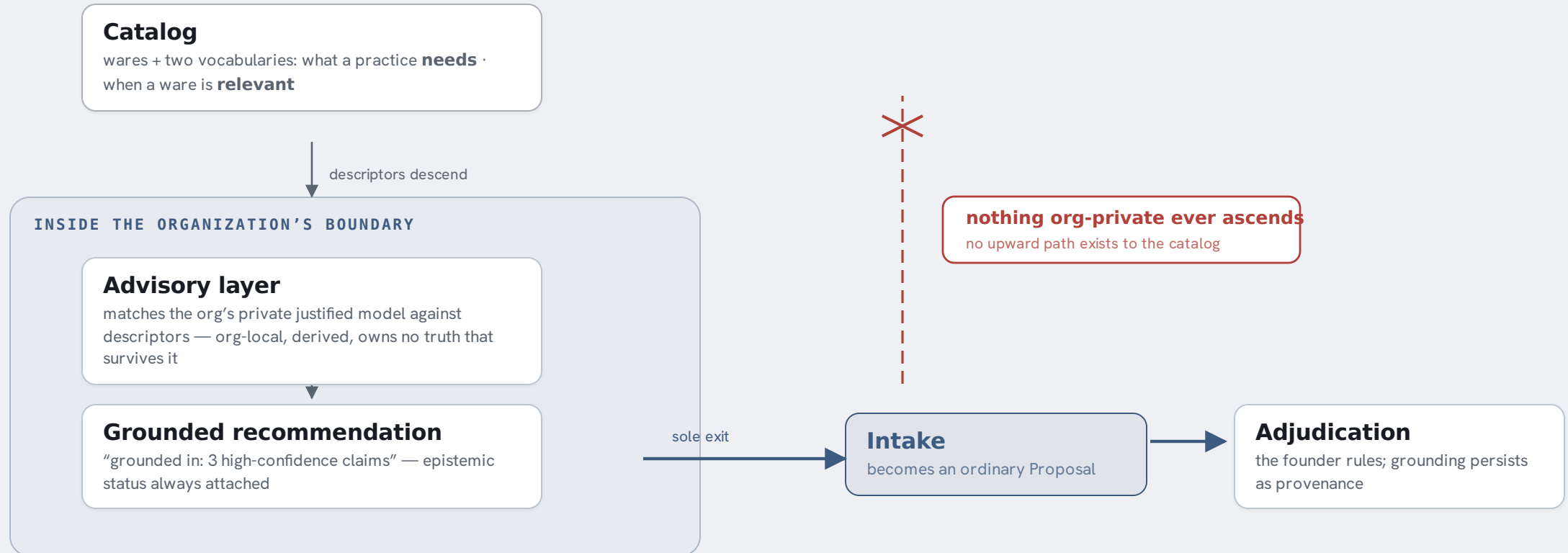
Marketplace distributes.

third-party publishers, typed trust
regimes — when a market exists

○ VISION

Catalog before Marketplace: with one publisher it is a library, and naming it honestly keeps engineering pointed at the real problem — the matching vocabulary, not storefront machinery.

The recommendation pipeline



The hard law: the catalog descends; the constitution never ascends. Zero cross-org data by default — "organizations like you" filtering is architecturally forbidden without its own opt-in ruling.

No constitutional bypasses

Advice is evidence, never authority.

Its only door is Intake; a recommendation that skips the pipeline is authority smuggling.

No platform auto-apply, ever.

Acceleration only by an org-ruled standing policy inside the org's own governance — and even that is treated as dangerous.

Publishers get identity, not reach.

Registration, trust regime, signing, provenance on every ware — built with one row (Diskarte) so it is load-bearing before third parties arrive.

Knowledge and substrate are typed apart.

Executable wares need signing and review that knowledge content does not. One storefront perhaps; never one trust model.

Standards are yardsticks, not wares.

External reference kinds you cite and are measured against — never forked, never owned; conformance is derived.

The organization can always leave.

The constitutional export right: adopted capabilities, derived practices, and the journal, in readable form. Ruled now; built with the first real customers.

Zero new constitutional primitives. The entire catalog era adds principals and provenance — never a new authority. That is what “Alpha is closed” looks like under adversarial review.

What an organization knows must never depend on what its software runs on.

Execution abstraction, provider abstraction, provenance, and the managed appliance.

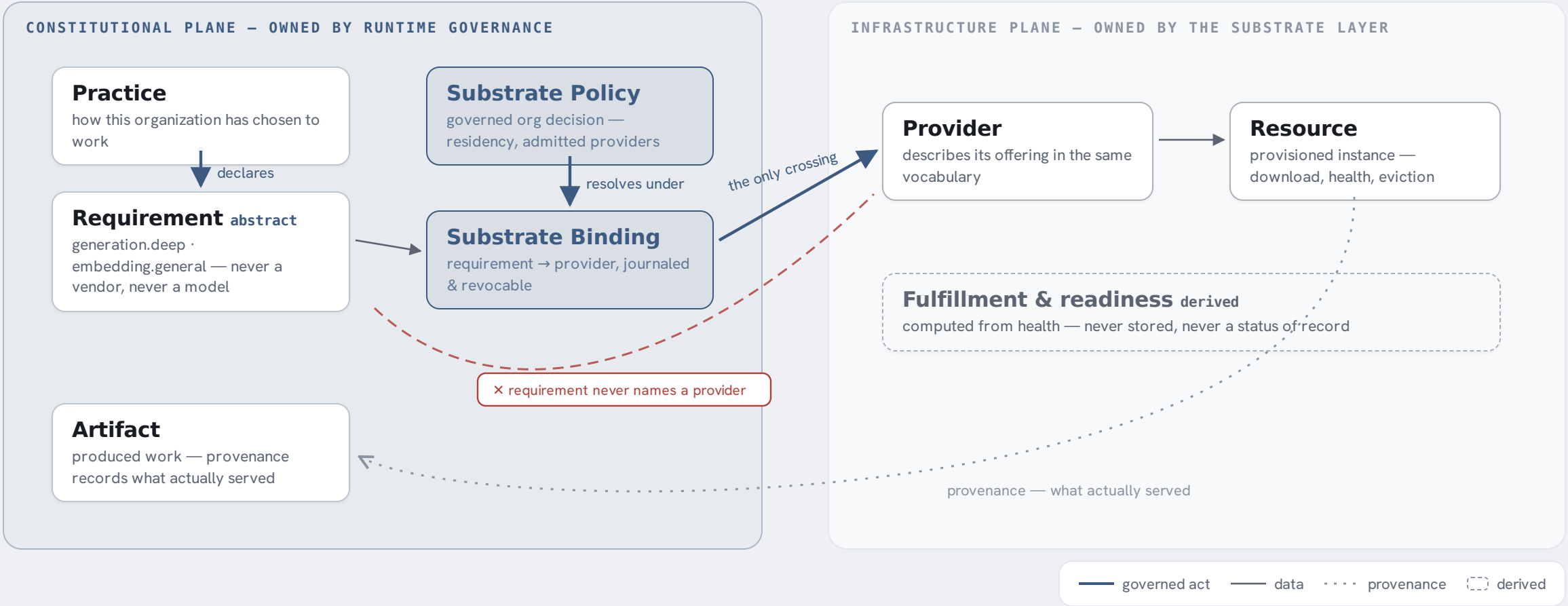
Desktop · Console

Runtime
constitutional truth

Cognition
· Substrate

Operations Platform · Cloud

The substrate seam



This seam exists because replacing a provider must never mutate a Practice. If replacing X changes what the organization knows how to do, X was capability; if only how well or where — X is substrate.

Requests never reference models

IN: AN ABSTRACT PROFILE

generation.fast
generation.deep
generation.private
embedding.general
extraction.structured
reranking.general

A fixed, versioned vocabulary — the database schema refuses anything else. A vendor or model name can never be written into the constitution. New worlds extend by adding a value, never by smuggling a product name.

OUT: PROVENANCE

At serve time, the cognition interior resolves the profile to whatever resource the organization's bindings admit — and the produced record names **what actually served**: model, provider kind, resource — including fallbacks.

Pin in the record, never in the requirement. Reproducibility is evidence on the artifact — not a contract that freezes the organization to a vendor.

Model identity flows in exactly one direction: out. Enforced by guard tests — a brief that addresses a model is a build failure, not a code smell.

Five profiles, one seam

PROFILE	POSTURE	WHO HOSTS · WHO HOLDS CREDENTIALS	STATUS
Local	on-device only; nothing leaves the boundary — the sovereignty default	the organization · none needed	● IMPLEMENTED
Diskarte Managed	org-owned appliance on Diskarte-operated compute; an explicit, adjudicated residency choice	Diskarte operates · Diskarte-held	● IMPLEMENTED PRODUCTION DEPLOYMENT PENDING
Hybrid	local + cloud APIs split by sensitivity class	mixed · org-held keys	○ ARCH. COMPLETE
Cloud	the org’s own cloud endpoints	the organization · org-held	○ ARCH. COMPLETE
Air-Gapped	local only, offline fulfillment via signed bundles	the organization · none	○ PLANNED DEFERRED BY RULING

A profile is a **published, versioned template** — not organizational state. Adopting one decomposes into an org-owned policy plus bindings; the later profiles are new catalog values, added without architecture change.

Migration is a governed policy amendment, never a restart. Managed → Hybrid re-resolves bindings underneath an unchanged organization: practices, capabilities, and canon never move.

The managed appliance

EACH ORGANIZATION OWNS

Deployment — the appliance

exactly one per organization; the isolation partition for every read

Versioned manifest

model mappings, compute backend — a new revision is cut, never edited; rollback and provider migration are pointer moves

Organizational state

routing, memory, adapters, data — **never multiplexed**

DISKARTE OPERATES — SHARED

Scheduler

a pure lifecycle machine over opaque appliance handles — it never sees organizational cognition

Ephemeral compute

workers under a warm-capacity cap; the backend is a manifest field — swappable, never a rewrite

APPLIANCE LIFECYCLE

Cold → Warm → **Running** → Sleeping → Evicted

Idle is storage, not compute. Waking at capacity evicts the oldest *evictable* resident — never one that is serving. When everyone is serving, the wake is refused with honest backpressure, never a silent drop. An evicted appliance keeps its manifest and re-warms.

Share utilization, never share cognition. Compute is replaceable and multiplexed; the organization's intelligence substrate is durable, isolated, and permanently its own.

Execution records: honest metadata, durably

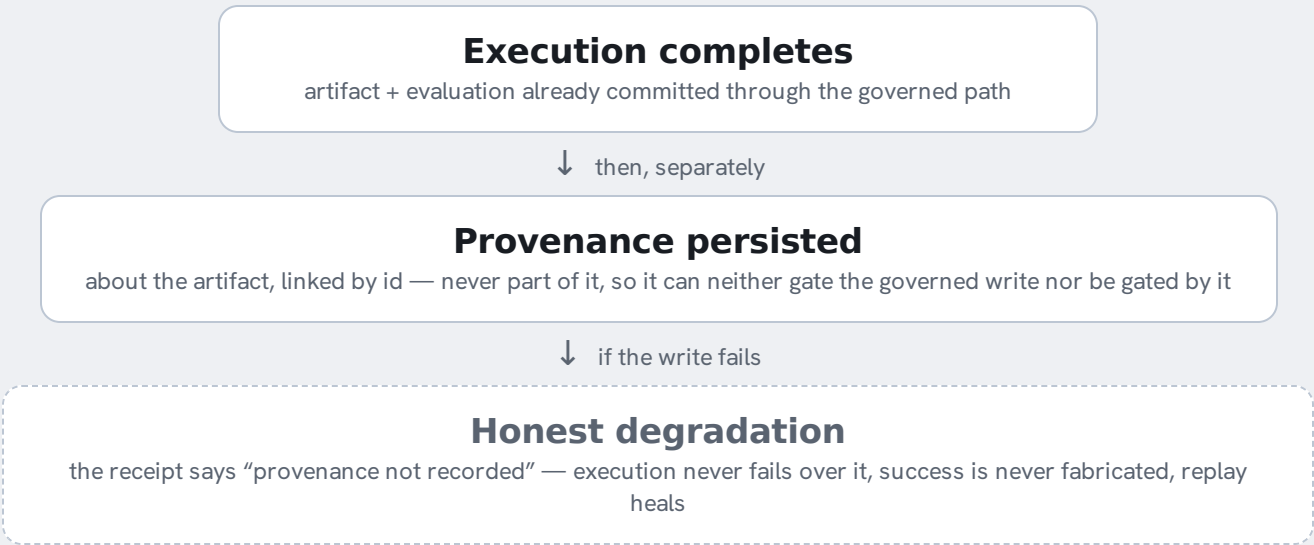
Every machine-produced artifact carries a durable record of what produced and what judged it.

Recorded: model identity, abstract profile, provider kind, resource reference, offered grounding, ungroundedness, requested language.

Never recorded: secrets, endpoints, keys, or reasoning bodies.

● IMPLEMENTED

RATIFICATION PENDING



Evidence must never sound like authority. Provenance is evaluated by nothing — it answers questions; it gates nothing.

Who owns what, below the constitution

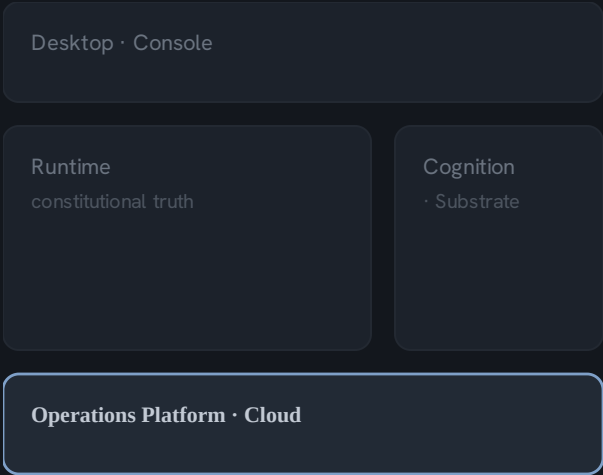
OBJECT	TRUTH OWNER	CLASS
Requirement (abstract)	Runtime — part of the practice’s contract	constitutional declaration
Substrate policy & bindings	Runtime governance	governed decisions — journaled, revocable
Profile catalog	the platform (published data)	versioned templates; adoption provenance is the only trace
Providers & resources	the substrate layer	infrastructure truth — lifecycle, health, credentials behind the secrets seam
The managed appliance	the organization (operated by Diskarte)	operational resource — never organizational state, never another org’s
Fulfillment / readiness	nobody stores it	derived from health, computed on read
Provenance	the produced record	evidence — evaluated by nothing

Air-Gapped honesty: offline fulfillment needs signed bundles, which collides with the platform’s minimal-secret posture. That exception will be ruled when a sovereign deployment is a committed target — pre-recorded now so it is a decision then, never an accident mid-build. Until then the profile is shown as unavailable, with the reason.

Someone has to run the company. It must not be the constitution.

Provisioning, releases, installations, and appliances — owned outside the Runtime, structurally unable to govern any organization.

YOU ARE HERE



Why Operations lives outside the Runtime

TWO ROLES, TWO SCOPES

A **Platform Operator** is platform-scoped: provisions, observes, deploys, diagnoses, recovers. A **Founder** is organization-scoped: the root of one organization's authority.

The same human may hold both today. The roles never merge — because at scale they will not be the same human.

STRUCTURAL, NOT POLICY

The Operations Platform's database access rides one seam with a hard allowlist — it is **structurally incapable** of opening a Runtime store.

Company history enters an organization only as an export bundle through the Runtime's own import door — a founder act, never a platform write.

Ratified law, all profiles: Diskarte-the-vendor holds no authority, read, or write inside your organization — structurally, not as policy. Your organization has exactly one root: you.

The operations plane

OPERATIONS PLATFORM — ENABLING STATE ONLY

Operators

every write attributed; issuance forms a lineage from operator zero

Releases & artifacts

versioned, digest-pinned; cloud deploys digests, never source

Installations

registered instances: desktop, cloud, on-prem — liveness reflected, never asserted

Appliances

the managed substrate control plane

Organization directory

a routing directory — when it disagrees with a Runtime, the Runtime wins

Deployments

what runs where — recorded software deploys

Setup codes

single-use, expiring, hash-at-rest; secret-free payloads; consumption is exactly-once

Journal

append-only, attributed; provisioning “runs” are a projection over it — no state machine

Operations Console

operator SPA — speaks only the operations contract, holds no org authority

Identity

operator sign-in via the configured issuer; verified subject → operator binding; fail-closed

Deployment discipline

CI holds no cloud credentials; every apply is an explicit human act

Ceremony state is derived here too: each provisioning step emits an attributed journal event; the “run” view is a filtered read, never an orchestrator of record.

Provisioning, with no shortcuts



The end-to-end legitimate path — **no SQL, no seed data, no hidden flags, no founder-only shortcuts.** Diskarte itself becomes Organization #1 through this same door: the company eats the ceremony it sells.

● IMPLEMENTED

Proven end-to-end against the real stack; the public claim surface ships with the next deployment gate.

What is actually running today

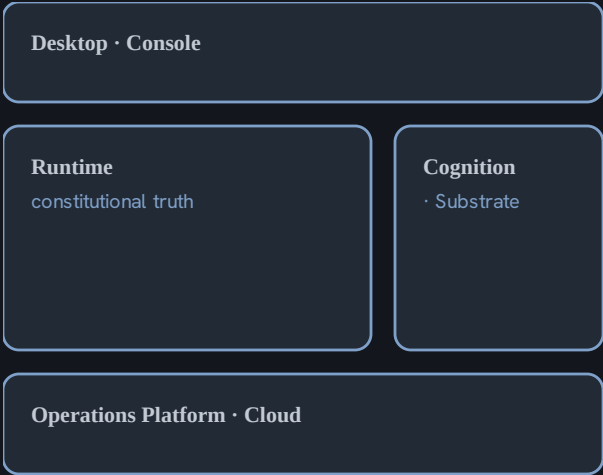
COMPONENT	WHERE	STATUS
Operations API + database	GCP Cloud Run + managed Postgres — IAM-gated, no public access, digest-pinned deploys	● DEPLOYED ● INTERNAL DEV
Operations Console + operator identity	static SPA behind an identity-aware edge; Auth0 as the first configured issuer	● DEPLOYED ● INTERNAL DEV
Claim gateway (the one public door)	written and proven locally; cloud deployment is the next gate	● IMPLEMENTED FOUNDER-GATED
Managed appliance control plane	proven against the test stack; not deployed	● IMPLEMENTED
Managed worker infrastructure	infrastructure code validated; first apply + billing are explicit human acts	● IMPLEMENTED DEPLOYMENT-READY

GCP and Cloud Run are the **current** deployment providers — implementations behind the infrastructure seam, not architectural dependencies. The runtime itself remains local-first; nothing above requires it to be hosted.

Every credential boundary is human-shaped: CI validates and refuses; only a person applies, spends, or licenses.

The honest ledger.

What is proven, what is deployed, what is ruled-but-unbuilt, and what is deliberately not done yet.



Subsystem status

SUBSYSTEM	STATUS	NOTES
Kernel · governed loop · gates	● IMPLEMENTED	live-proven end to end; guard-enforced purity
Identity · membership · sessions	● IMPLEMENTED	real-tenant identity proof; provider claims never become authority
Morning Brief · Memory · Judgment	● IMPLEMENTED	judgment’s probabilistic synthesis deliberately parked
Claim ceremony + onboarding path	● IMPLEMENTED	public deployment founder-gated; selection screen is presentation
Substrate seam · provenance · appliance	● IMPLEMENTED	ratification pending — tracked, not drifting
Brownfield import (offline half)	● IMPLEMENTED	live connectors gated by a recorded ruling
Operations Platform	● DEPLOYED · INTERNAL DEV	the only deployed footprint, by intent
Foundations · Catalog · advisory layer	○ ARCH. COMPLETE	first advisory tick implemented (profile recommendation)
Delegation charter · export right	○ ARCH. COMPLETE	implementation explicitly evidence-gated
Hosted Runtime · Hybrid/Cloud profiles	○ PLANNED	gates recorded; export bundle precedes hosting
Marketplace · publisher ecosystem	○ VISION	no work before a second organization lives

Full per-repository detail, the ruling inventory, and the realization matrices live in the **appendix**.

Stated plainly

Zero external organizations have used Diskarte.

All usage evidence is internal dogfooding. The one simulated “founder month” was operated by the same intelligence being evaluated — it is adversarial testing that found and fixed real defects, not customer validation, and it is never counted as such.

Managed production deployment has intentionally not happened.

The worker and its infrastructure are written, validated, and deploy-ready. No production license accepted, no billed infrastructure created — those are deliberate human gates, not gaps.

The deployed footprint is the internal Operations Platform, in development GCP.

Serving Diskarte-the-company. Nothing customer-facing is production-deployed.

The Greenfield gate is implemented and proven — internally.

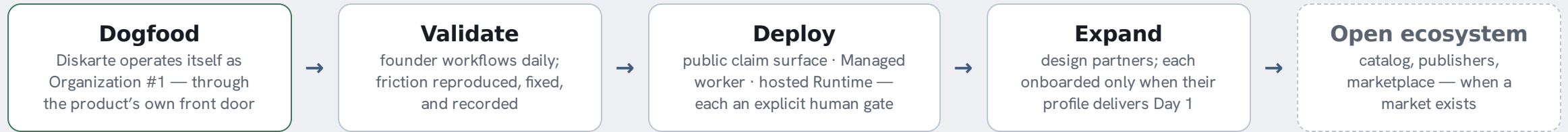
Every engineering milestone shipped with its verification green across all repositories. Some implementation currently precedes canonical ratification; Diskarte tracks that debt explicitly rather than allowing silent architectural drift.

The honest gaps beyond deployment: the deep execution domains (production, operations of the org’s own products), the visual onboarding screen, desktop signing/updater credentials, and multi-actor governance behind its privacy charter.

Gates, not dates.

Progress is eliminated uncertainty, never elapsed time. Every step below names the gate that opens it.

The sequence



The next gate is always **whichever decision the next real organization needs** — customer evidence outranks any pre-written order. Nothing on this page carries a quarter or a calendar promise.

Near-term and later

NEAR-TERM — THE ALPHA LOOP

Dogfooding — the company inside its own product, daily

Founder workflows & UX polish — the onboarding screen, desktop refinement, Morning Brief maturation

Managed deployment — first worker apply, model backend licensing, public claim surface

Hosted Runtime — export bundle first (the exit door before anyone moves in), then per-org isolation

LATER — THE CATALOG ERA

Catalog — foundations, capability packages, profiles as published, versioned wares

Foundation library & recommendation breadth — the advisory layer running continuously, not just at bootstrap

Publisher ecosystem → Marketplace — third parties, typed trust regimes, signing

Deeper delegation — the first evidence-backed machine trust grants, each with a regression rule

Deliberately refused, permanently: platform auto-apply · cross-org learning by default · regional runtime forks · authority without a human ratification behind it.

Less an AI application. More an operating system.

Where this is going, and who is building it.

Diskarte was built by someone who kept watching organizations lose the plot.

- 01 **Self-taught engineer.** No credential stood between her and production software — the discipline came from shipping.
- 02 **Production software engineer.** A frontend specialist who spent years inside large production systems — modernizing them while they served, across engineering cultures on both sides of the world.
- 03 **The same failure, everywhere.** Decisions lost their rationale; context died in handoffs; every system knew its tasks and none knew its organization. The problem was never the tools.
- 04 **Founder.** When AI made execution nearly free, the missing layer stopped being an annoyance and became the bottleneck of everything. She left to build it.
- 05 **Building Diskarte.** Founder and principal engineer: the architecture in this deck is ruled, built, and dogfooded by the person presenting it.



Margaux Flores

FOUNDER & PRINCIPAL ENGINEER

Organizations will need an operating system.

Not another assistant, and not a smarter agent — a runtime that makes an organization legible to machines without surrendering it to them: authority computed, knowledge justified, canon human-ratified, execution accountable.

Diskarte is built as that operating system. Today it is a runtime a founder can trust; the trajectory — founder's runtime, organizational runtime, ecosystem — is gated by charters and evidence, never by rewrites. The boundaries you have seen in this deck are the ones that will still be here in ten years.

Diskarte

Move at AI speed without losing the plot.

Design-partner alpha — by application, deliberately small.

`diskarte.jp`

Technical diligence.

Milestone glossary, decision-record index, ratification debt, repository detail, terminology reconciliation, protocol reference, and realization matrices. Internal identifiers appear here by design; the core narrative avoids them.

Milestone & program glossary

NAME	MEANING	OUTCOME
M0 / M1	Kernel constitution (seven primitives), then the first runtime realization: governed loop, Platform API, protobuf contracts, Postgres adapter, local-first topology	Implemented; kernel constitution unchanged since ratification
R2	Cognition Runtime: the cognition domain boundary (Produce/Judge port, interior/shell split, boundary guards)	Accepted + code-complete
C0 / C0A	Platform scope resolution; identity, membership, and authority spine	Implemented
DC-16	The Delegation Charter — one root lineage, Declared Reach, Ratification–Allocation, custody law (15 rulings)	Ruled; implementation evidence-gated
OP-1 / OP-2	The Operations Platform program: charter + authority separation, then OIDC identity, installations, setup-code coordination, operator console, and the end-to-end provisioning ceremony	Implemented; deployed to internal dev GCP
PC-1	The identity-bound claim ceremony program: claim contract, gateway, vocabulary rulings, desktop ceremony; deployment gate remains	Implemented + proven; deployment founder-gated
GF-0	The Greenfield Alpha gate: substrate seam, execution provenance, managed appliance control + data plane, Environment Report, recommendation + adoption path, honesty codification	All engineering milestones shipped; founder-gated remainder

GF-0 milestone detail

MILESTONE	CONTENT	REPOSITORY	STATUS
M0	Architecture canon repository (Parts I-V, doctrine, decision register)	canon	committed; remote pending
M1	Substrate seam: org-scoped policy + bindings, abstract requirement vocabulary, schema-refused vendor names	runtime	shipped
M2	Durable execution provenance: producer + judge records, best-effort with honest degradation	runtime	shipped
M3	Managed appliance control plane: one appliance per org, versioned manifests, scheduler lifecycle, atomic quota	dev	shipped
M4	Managed worker data plane + serving seam; interior profile→resource resolution; deploy-ready cloud infrastructure	runtime · cloud	shipped; deploy founder-gated
M5	Environment Report: secret-free desktop machine probe with honest unknown-degradation	desktop	shipped
M6	Substrate Profile catalog + deterministic recommendation engine; adoption RPC; live desktop wiring	ui · runtime · desktop	shipped; visual screen pending
M7	Air-Gapped honesty codified — unavailability stated with its reason	ui	shipped

Founder-gated remainder: canon remote creation · production model backend + licensing · first cloud apply + billing · schema reset after new migrations · one test-environment dependency upgrade.

Selected architecture decision records

ADR	DECISION	STATUS
0003	Protobuf canonical at every boundary; kernel imports none of it	Accepted
0004	Platform API is the single frontend seam	Accepted
0005	Local-first runtime topology	Accepted
0006 / 0007	Storage adapter (managed Postgres); canonical vs retrieval memory	Accepted
0012–0016	Kernel-backed authority boundary; Decision primitive; governance loop; evaluation-gated completion	Accepted
0025	Brownfield reconstruction loop	Proposed — built ahead of ratification

ADR	DECISION	STATUS
0027	Five-port execution-adapter architecture (production sessions, repository/deployment/verification/telemetry ports). Historical: a fuller execution plane sketched early; the implemented execution seam (0029 + GF-0) took a different, narrower shape. Not current architecture.	Proposed — unbuilt
0028	The Go runtime is the cognition runtime; Python worker scaffolds retired	Accepted
0029	Cognition is a domain boundary (Produce/Judge, guard-enforced)	Accepted
0030 / 0031	Operations Platform charter; operations OIDC + installation primitive + setup-code coordination	Proposed — built + deployed (internal dev)
0032 / 0033 / 0034	Substrate seam · managed appliance control plane · durable execution provenance	Proposed — built + proven

“Proposed — built” is not drift: it is tracked ratification debt, listed on the next slide.

Ratification debt — law lagging proven implementation

ITEM	WHAT DISCHARGES IT
ADRs 0032 / 0033 / 0034 (substrate seam, appliance, provenance) — built and live-proven	founder ratification (or amend-then-ratify), one sitting
ADR 0025 (brownfield) — built, committed, demonstrated	ratify or amend-then-ratify
ADRs 0026 / 0027 — proposed, unbuilt	ratify or park explicitly
Organizational constitutional model (the 13 primitives) — load-bearing, formally unstatused	a status ruling; realization matrix in A10
Doctrine entries 1.19-1.22 + one §2 addition — intentionally pending	a ratify-or-strike sitting
Brief-fidelity confirmations for the authoring-brief corpus	founder signatures on the archived briefs

This register is the mechanism, not the problem: implementation may run ahead of canon **only** where the debt is recorded, and the ledger is the single place where “decided” is defined. A blank row is the failure class — if it is not in the ledger, it is not decided.

Repository implementation detail

REPOSITORY	OWNS	PUBLISHES	STATE
runtime	kernel, contracts, Go runtime, constitutional corpus	TS/Py contract packages, capability packages, release binaries, OCI images	implemented + live through GF-0
ui	console, design system, localization, platform client	versioned UI artifact (hash-manifested, byte-deterministic)	implemented through GF-0; onboarding screen pending
desktop	Tauri host, custody, keychain, claim ceremony, composition manifest	the desktop app (signing/updater inactive pending credentials)	ceremony proven incl. real-tenant identity
dev	Operations API + CLI, ops schema, claim gateway, appliance control plane	operations service image	deployed (internal dev); gateway + appliance built, not deployed
cloud	Terraform roots, IAM, budgets, deploy/rollback scripts	—	operations root applied; worker infra validated, not applied
cognition	advisory research: prompts, evaluation, calibration, training	advisory proposals (future)	deliberate skeleton — chartered before its first model line
canon	ruled architecture, doctrine, decision register	—	approved through Greenfield closure
site	public surface (diskarte.jp)	—	live

Federation law: every cross-repo dependency is a versioned published artifact — building another repository’s source *is* a source import, and is refused.

Terminology reconciliation

SUPERSEDED / COLLIDING	CANONICAL	WHY
“Strategic Brief”	Morning Brief	early design-mock vocabulary; one surface, one noun
“challenge”	claim ticket	the desktop flow already carries a PKCE code-challenge; two “challenges” in one ceremony is a defect factory
“IssuanceContext”	IssuanceChannel	“context” sounds evaluable; channel values are evidence, evaluated by nothing
“bootstrap package” (customer-facing)	setup code	protocol nouns never reach customers
“Intent” (as an advisory object)	deleted	organizational direction lives as evidenced claims; the word belongs to the ceremony’s confirmation gate only
“Advisory Plane”	advisory layer	a plane owns truth; advice must own none — its sole exit is Intake
“Marketplace” (before third parties)	Catalog	a market without third parties is a library; the name pulls engineering toward storefronts
Deployment (software release) vs Deployment (appliance)	release deploy · appliance	two objects, deliberately named apart in code; prose always says “appliance” for the latter
“Hiraya” (runtime actor)	a UI projection only	the intake persona never became a runtime actor

Deployment-profile model: historical evolution

An earlier strategy ruling defined four **Alpha Deployment Profiles** sliced by cognition source on a local runtime. The Greenfield architecture superseded that frame with the five **Substrate Profiles** — sliced by how cognition is served, decomposing into governed policy at adoption. The mapping:

ALPHA PROFILE (HISTORICAL, DC-28)	SUBSTRATE PROFILE (CANONICAL)	NOTE
Hosted Trial	Diskarte Managed	generalized from “managed cognition provider” to the org-owned appliance model
Sovereign	Local (and later Air-Gapped)	Local is the sovereignty default; Air-Gapped adds offline fulfillment behind the deferred signing ruling
BYO-AI	Hybrid	customer keys behind the secrets seam, split by sensitivity class
Hybrid (old sense)	Hybrid / Cloud	the old name conflated key-custody and infra-custody; the new pair separates them
—	Cloud (BYO infra)	new value: the organization’s own endpoints and credentials

One ruling survived the transition unchanged: a profile is configuration over the same primitives — never a fork of the runtime.

Claim protocol reference

AXIS	VALUES	LAW
ClaimProtocol GATE	identity_bound · legacy_possession (starving)	names the consumption law; cross-protocol calls are structurally dead; new legacy issuance is refused entirely
ClaimPolicy AUTHORIZATION	authenticated_holder · subject_bound · (reserved: email_bound, domain_bound, organization_invite, approval_required)	evaluated inside the claim transaction, never client-side; each reserved mode gated on its own data-minimization ruling
IssuanceChannel EVIDENCE	operator_assisted · self_service · partner_admin	evidence of the distribution channel — evaluated by nothing, ever

Amplification bounds

one active ticket per claimant per code (self-supersede) · three per code · ~10-minute expiry, self-healing · completed tickets retained as evidence · failed attempts leave nothing durable

Non-enumeration

every wire refusal is one byte-identical dead answer; honesty flows through the desktop (its own state) and the issuer’s console (the record) — never the wire

Extension law

new worlds (enterprise, machines, sovereign) extend by axis value, never by branch — an if for a new world inside an existing law is the failure signal

Current provider implementations

SEAM	CURRENT IMPLEMENTATION	BOUNDARY DISCIPLINE
Identity	Auth0 (first configured OIDC issuer)	issuer/audience are configuration; provider claims never become authority; Diskarte-owned sessions are the record
Deployment	GCP Cloud Run (+ managed Postgres)	infrastructure repo deploys immutable digests only; every apply is a human act; CI holds no credentials
Canonical storage	PostgreSQL behind the storage adapter	an adapter, not a privileged truth; migrations forward-only, hash-registered
Retrieval	Qdrant behind the retrieval port	derived and rebuildable; never a source of truth
Local model plane	Ollama over loopback	one bounded completion per call; deterministic stub as the honest floor
Managed serving	stateless worker behind the cognition port	backend is configuration (deterministic today; a model backend is a flip + a licensing decision)
Contracts	protobuf / Connect	canonical at every boundary; the kernel imports none of it

Every row is a **current implementation, not an architectural dependence** — each sits behind a port whose replacement requires no kernel change.

Organizational primitives — realization matrix

PRIMITIVE	MEANING	REALIZATION TODAY
Evidence · Proposal · Governance Readiness · Influence-vs-Authority	the epistemic substrate and its guardrails	fully realized
Genesis · Governed Canon · Drift · Distributed Consensus	founding, commitment, divergence, ratification	partially realized
Governance Policy · Drift Classification · Epoch	who may ratify what; the drift taxonomy; era boundaries	declared, not yet realized (policy hard-coded; taxonomy named; epoch stamped only)
Integrity · Constitutional Crisis	tamper resistance; what happens when governance itself fails	future research, deliberately

The source document is formally **unstated pending a ruling** — presented here exactly as such. The mechanism it describes (evidence → inference → proposal → ratification → canon) is implemented in the runtime’s verification and promotion paths regardless; the matrix above tracks the model’s own realization claims, per primitive, without inflation.

How “proven” is defined

Structural guards in code

import-direction tests enforce kernel purity, proto containment, and the cognition boundary; a violation is a build failure. “Guards are the architecture.”

Live proofs against real substrate

gated suites run the real database, real local model, real processes — including full-ceremony batteries that spawn issuer, service, gateway, and runtime together.

Cross-repository regression

the Greenfield gate closed with every repository’s full battery green, plus the desktop composition check assembling pinned artifacts with no source reachable.

Adversarial review as method

every major design passed a red-team pass whose findings are recorded documents; the founder-month simulation was run to break the product and is counted only as that.

End of appendix. The canonical corpus — doctrine, rulings, records, and per-repository state — remains the source of truth behind every claim in this deck.